Team California
Henrique Moreira(119809), João Cardoso(120440), Nicolas Sousa (119744)

Relatório do Lab 11

v2025-22-10

Introdução

Para compreender melhor o funcionamento de um sistema, é essencial saber pensar e visualizar a colaboração entre os seus diferentes componentes.

Neste trabalho, procuramos aprofundar e consolidar os conteúdos abordados em sala de aula, com especial foco na análise das interações entre partes de um sistema. O objetivo principal é aplicar esses conhecimentos em diversas situações práticas, desenvolvendo a capacidade de modelar, interpretar e representar o comportamento colaborativo dos elementos que compõem uma solução de software.

Distribuição de trabalho

Henrique : Atividade 2.
João : Atividade 1.
Nicolas : Atividade 3 e diagrama de sequências 2.

Referências e materiais consultados

Materiais e apontamentos disponibilizados pelo professor Ilídio Oliveira.

https://www.lucidchart.com/pages/es/diagrama-de-secuencia

Atividades

Atividade 1: Colaboração entre componentes de uma arquitetura

1. Identificação dos Módulos da Arquitetura

Com base nos requisitos funcionais descritos, identificam-se os seguintes componentes (módulos) lógicos da solução:

1. Portal VoipGest (WebApp/Frontend): Interface responsável pela interação com o utilizador (Responsável da Distribuição) e submissão dos dados.

2. API VoipGest (Backend): Componente central de lógica de negócio que orquestra os pedidos, valida regras e comunica com os restantes serviços.

3. Base de Dados de Operações (BD-Ops): Repositório de dados persistentes do sistema VoipGest (armazena as relações entre utilizadores e empresas).

4. Sistema de Autenticação (Cloud): Serviço externo/segregado responsável pela gestão de identidades, criação de credenciais e segurança.

5. Serviço de Email: Serviço utilitário responsável pelo envio de notificações assíncronas aos utilizadores.

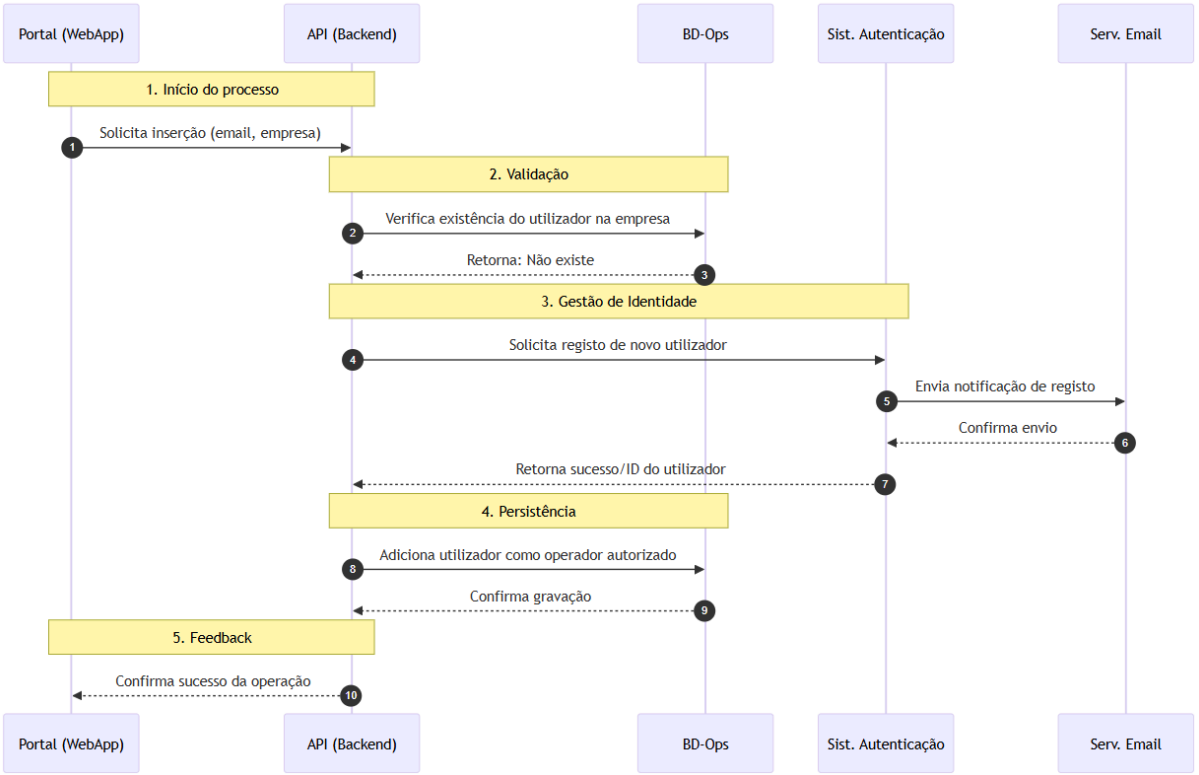


Figura 1: Diagrama de colaboração de módulos.

Descrição do Fluxo:

1. **Portal -> API:** O Frontend envia os dados do novo operador para o Backend.
2. **API -> BD-Ops:** A API consulta a base de dados para garantir que não há duplicados nessa empresa.
3. **API -> Auth:** Se o utilizador é novo, a API pede ao sistema de autenticação para criar as credenciais. Este, por sua vez, aciona o Serviço de Email.
4. **API -> BD-Ops:** Com o utilizador criado, a API grava a associação final na base de dados de operações.
5. **API -> Portal:** O sistema informa o interface de que o processo terminou com sucesso.

Atividade 2: Colaboração orientada a serviços

Cenário de Colaboração: Pagamento de Sessão de Carregamento

Contexto: O utilizador (Condutor) finalizou um carregamento num posto e procedeu ao pagamento do valor (ex: 15,50 €) a partir da App MultiPower.

Serviço Externo: API Stripe Protocolo: HTTP/1.1 (REST) Formato de dados: JSON

Em vez de gerir os dados sensíveis do cartão de crédito diretamente na tua base de dados (o que exigiria uma certificação PCI-DSS complexa), utilizamos a API da Stripe. O fluxo

moderno recomendado é o de PaymentIntents, que suporta a autenticação forte (SCA) exigida na Europa (Portugal).

Endpoints da API (Exemplo)

Para que isto funcione, o sistema interagirá principalmente com estes recursos:

Criar Intenção de Pagamento (Backend para Stripe):

Método: POST

URL: https://api.stripe.com/v1/payment_intents

Função: O servidor diz à Stripe: "Vou cobrar 15,50 EUR". A Stripe responde com um "segredo" (client_secret).

Confirmar Pagamento (App Móvel para Stripe):

SDK Móvel: Função confirmPayment

Função: A App envia os dados do cartão e o client_secret diretamente para a Stripe. A Stripe valida e processa a cobrança.

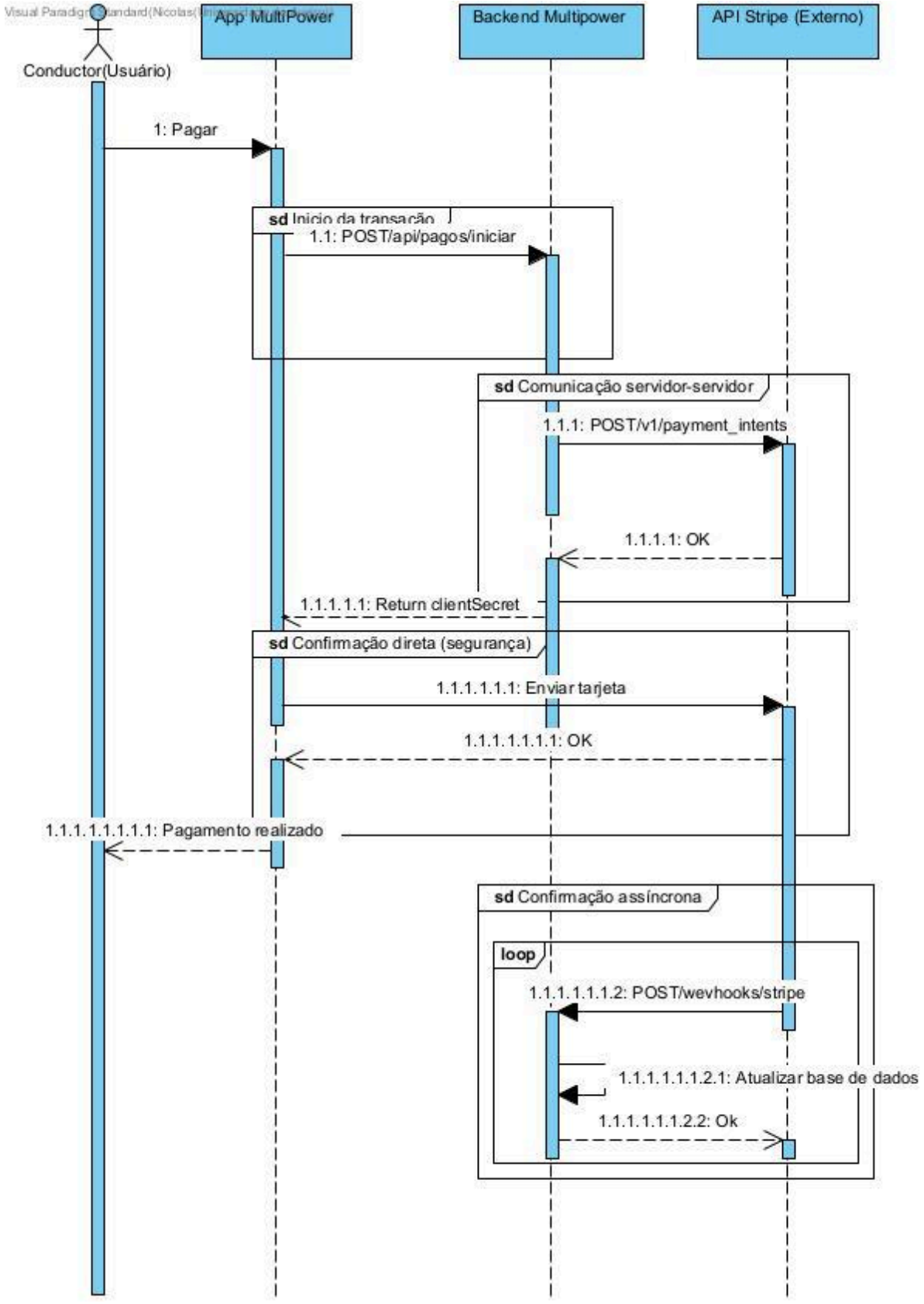


Diagrama de Sequência (UML) 2 - Este diagrama mostra como interagem o Utilizador, a Aplicação (Frontend), o Servidor (Backend MultiPower) e a API Externa (Stripe).

Segurança (Não tocar no cartão): O Servidor MultiPower nunca recebe o número do cartão do utilizador. A App envia esses dados diretamente para a Stripe (Passo 3). Isto reduz drasticamente o risco de segurança para o nosso projeto.

O client_secret: O backend obtém-no da Stripe e passa-o para a App. É como uma "permissão temporária" para finalizar essa transação específica.

Webhooks : Embora a App diga "Pagamento bem-sucedido", o servidor real (Backend) deve esperar que a Stripe o avise oficialmente através de um Webhook (uma chamada automática da Stripe para o teu servidor) para marcar a sessão como paga na base de dados. Isto evita fraudes se alguém manipular a App móvel.

Atividade 3: Colaboração entre objetos (de código)

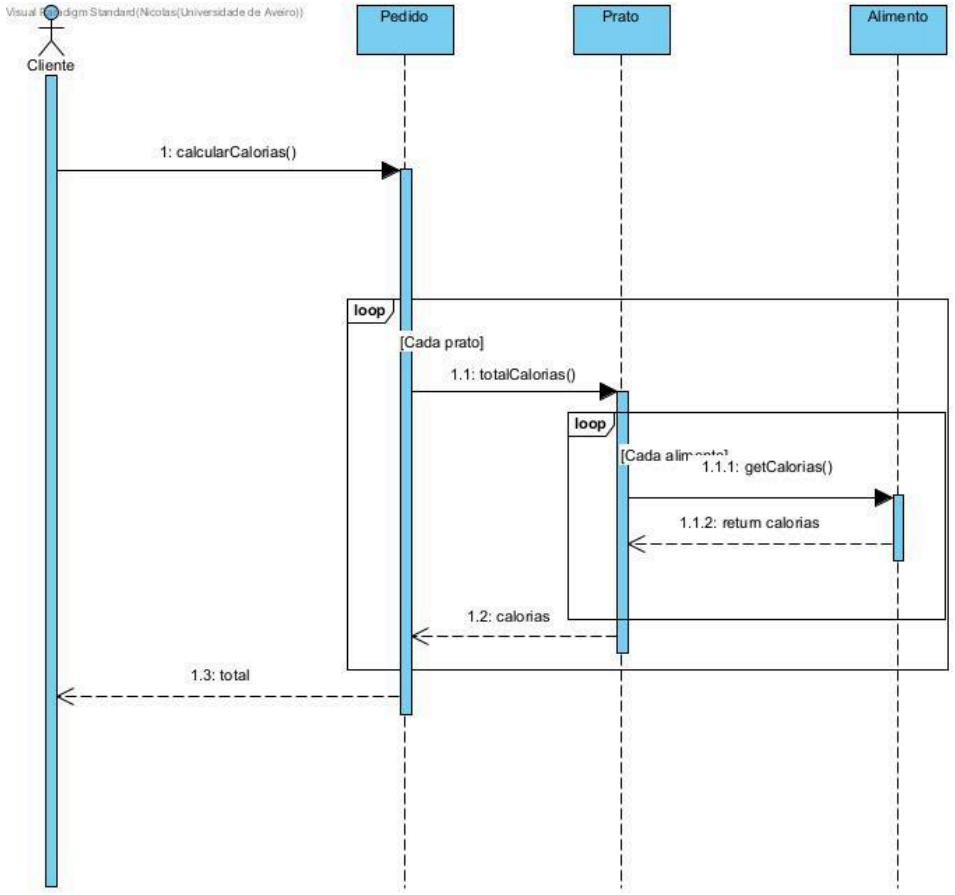


Diagrama de sequência 3 - Funcionamento da função calcularCalorias()

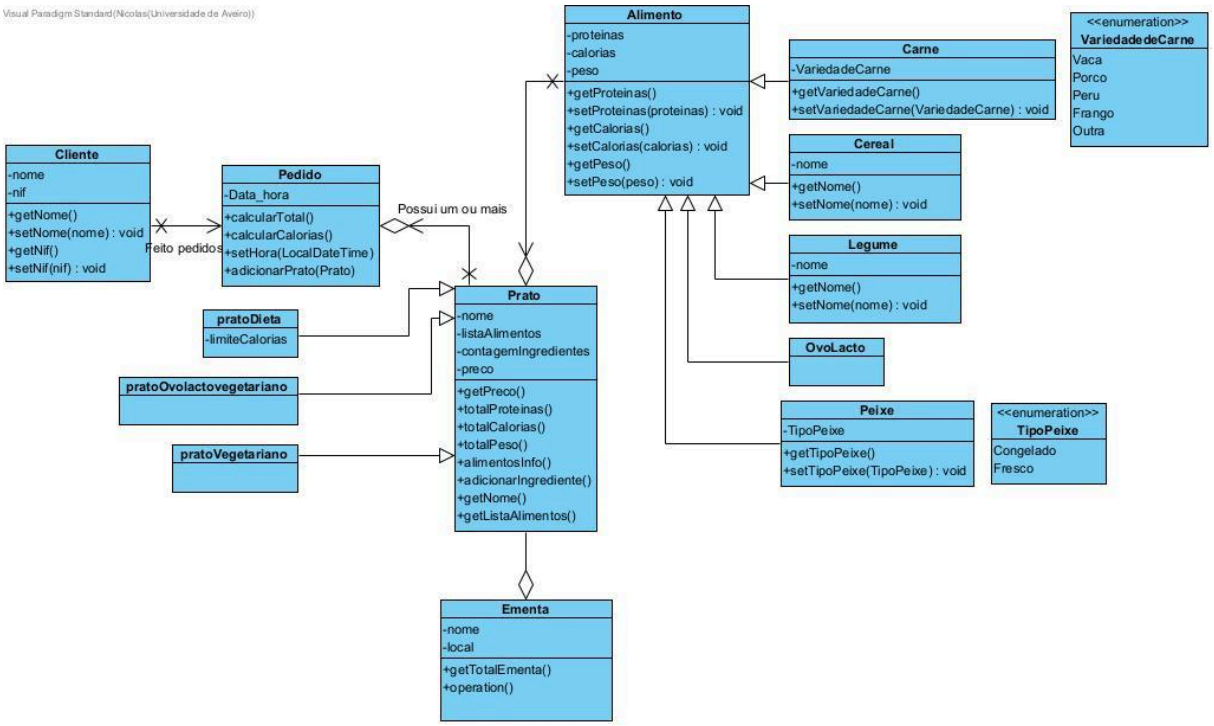


Diagrama de classe 4 - Classes do sistema restaurante.